

# Python Programming Fundamentals

Variables, Control Flow, Functions & Data — With Runnable Examples

---

Sample coursework material — for product demonstration purposes.

## Chapter 1 — Variables and Data Types

A variable is a named reference to a value stored in memory. Python is dynamically typed: a variable's type is determined by the value assigned to it, and can change if a new value is assigned.

```
name = "Ada"
age = 30
height = 1.68
is_student = False

print(type(name))
print(type(age))
print(age + 5)
```

Running this prints <class 'str'>, then <class 'int'>, then 35 — age + 5 adds 5 to the integer 30.

## Chapter 2 — Conditionals

if/elif/else statements let a program branch based on a condition. Python evaluates each condition top to bottom and runs the first block whose condition is True.

```
score = 72

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"

print(grade)
```

Since score is 72, the first two conditions (90, 80) are False, the third (score >= 70) is True, so grade is set to "C" and that is what gets printed.

## Chapter 3 — Loops

A for loop repeats a block of code once for each item in a sequence. The range() function generates a sequence of numbers, commonly used to repeat a fixed number of times.

```
total = 0
for i in range(1, 6):
    total += i

print(total)
```

`range(1, 6)` produces 1, 2, 3, 4, 5 (the stop value is exclusive). Summing them gives  $1+2+3+4+5 = 15$ , so this prints 15.

```
count = 0
while count < 3:
    print("lap", count)
    count += 1
```

A while loop repeats as long as its condition stays True. This prints "lap 0", "lap 1", and "lap 2", then stops once count reaches 3.

## Chapter 4 — Functions

A function is a reusable, named block of code defined with `def`, optionally taking parameters and returning a value with `return`.

```
def square(n):
    return n * n

def sum_of_squares(a, b):
    return square(a) + square(b)

print(sum_of_squares(3, 4))
```

`square(3)` is 9 and `square(4)` is 16, so `sum_of_squares(3, 4)` returns  $9 + 16 = 25$ , which is what gets printed.

## Chapter 5 — Lists and Indexing

A list is an ordered, mutable collection. Elements are accessed by a zero-based index — the first element is index 0 — and negative indices count from the end, with -1 referring to the last element.

```
fruits = ["apple", "banana", "cherry", "date"]

print(fruits[1])
print(fruits[-1])
print(fruits[1:3])
```

fruits[1] is "banana" (index 1, second item). fruits[-1] is "date" (the last item). fruits[1:3] is a slice containing indices 1 and 2 (3 is exclusive): ['banana', 'cherry'].

## Chapter 6 — String Manipulation

Strings support many built-in methods for transforming and inspecting text, and f-strings let variables be embedded directly inside a string literal.

```
name = "prism"
greeting = f"Hello, {name.upper()}!"

print(greeting)
print(len(greeting))
```

name.upper() converts "prism" to "PRISM", so greeting becomes "Hello, PRISM!" — 13 characters long, which is what len(greeting) returns.